

基本概念

机器学习

机器学习 (Machine Learning, ML) 就是让计算机从数据中进行自动学习, 得到某种知识 (或规律)。

首先我们以一个生活中的例子来介绍机器学习的一些基本概念: 样本、特征、标签、模型、学习算法等. 假设我们要到市场上购买芒果, 但是之前毫无挑选芒果的经验, 那么如何通过学习来获取这些知识?

首先, 我们从市场上随机选取一些芒果, 列出每个芒果的**特征** (Feature), 包括颜色、大小、形状、产地、品牌, 以及我们需要预测的**标签** (Label). 标签可以是连续值 (比如关于芒果的甜度、水分以及成熟度的综合打分), 也可以是离散值 (比如“好”“坏”两类标签). 这里, 每个芒果的标签可以通过直接品尝来获得, 也可以通过请一些经验丰富的专家来进行标记.

我们可以将一个标记好**特征**以及**标签**的芒果看作一个**样本** (Sample), 也经常称为**示例** (Instance)。

一组样本构成的集合称为**数据集** (Data Set)。一般将数据集分为两部分: 训练集和测试集. **训练集** (Training Set) 中的样本是用来训练模型的, 也叫**训练样本** (Training Sample), 而**测试集** (Test Set) 中的样本是用来检验模型好坏的, 也叫**测试样本** (Test Sample)。

我们通常用一个 D 维向量 $\mathbf{x} = [x_1, x_2, \dots, x_D]^T$ 表示一个芒果的所有特征构成的向量, 称为**特征向量** (Feature Vector), 其中每一维表示一个特征. 而芒果的标签通常用标量 y 来表示.

机器学习方法可以粗略地分为三个基本要素:

- **模型**
 - **神经网络**、决策树、支持向量机、KNN、朴素贝叶斯、线性回归、逻辑回归、...

- **学习准则**

机器学习中的学习准则并不仅仅是拟合训练集上的数据, 同时也要使得**泛化错误最低**. 给定一个

训练集，机器学习的目标是从假设空间中找到一个泛化错误较低的“理想”模型，以便更好地对未知的样本进行预测，特别是不在训练集中出现的样本。

- 平方误差
- 交叉熵
- 过拟合、欠拟合

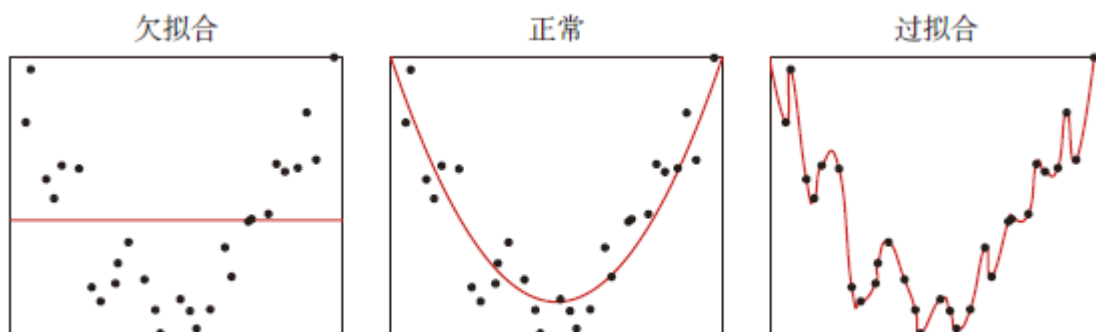


图 2.3 欠拟合和过拟合示例

• 优化算法

- 梯度下降
SDG, Adam, RMSprop, AdamW, ...
- 反向传播
- 提前停止(early stopping)

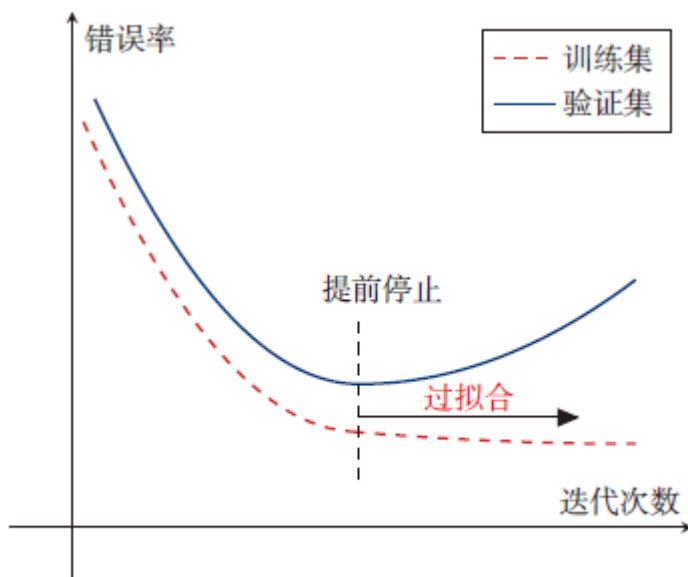


图 2.4 提前停止的示例

机器学习的工作流程可以大致分为以下几个步骤：

1. 数据收集

- **收集数据**：这是机器学习项目的第一步，涉及收集相关数据。数据可以来自数据库、文件、网络或实时数据流。
- **数据类型**：可以是结构化数据（如表格数据）或非结构化数据（如文本、图像、视频）。

2. 数据预处理

- **清洗数据**：处理缺失值、异常值、错误和重复数据。
- **特征工程**：选择有助于模型学习的最相关特征，可能包括创建新特征或转换现有特征。
- **数据标准化/归一化**：调整数据的尺度，使其在同一范围内，有助于某些算法的性能。

3. 选择模型

- **确定问题类型**：根据问题的性质（分类、回归、聚类等）选择合适的机器学习模型。
- **选择算法**：基于问题类型和数据特性，选择一个或多个算法进行实验。

4. 训练模型

- **划分数据集**：将数据分为训练集、验证集和测试集。
- **训练**：使用训练集上的数据来训练模型，调整模型参数以最小化损失函数。
- **验证**：使用验证集来调整模型参数，防止过拟合。

5. 评估模型

- **性能指标**：使用测试集来评估模型的性能，常用的指标包括准确率、召回率、F1 分数等。
- **交叉验证**：一种评估模型泛化能力的技术，通过将数据分成多个子集进行训练和验证。

6. 模型优化

- **调整超参数**：超参数是学习过程之前设置的参数，如学习率、树的深度等，可以通过网格搜索、随机搜索或贝叶斯优化等方法来调整。
- **特征选择**：可能需要重新评估和选择特征，以提高模型性能。

7. 部署模型

- **集成到应用**：将训练好的模型集成到实际应用中，如网站、移动应用或软件中。
- **监控和维护**：持续监控模型的性能，并根据新数据更新模型。

8. 反馈循环

- **持续学习**：机器学习模型可以设计为随着时间的推移自动从新数据中学习，以适应变化。

9. 技术细节

- **损失函数**：一个衡量模型预测与实际结果差异的函数，模型训练的目标是最小化这个函数。
- **优化算法**：如梯度下降，用于找到最小化损失函数的参数值。
- **正则化**：一种技术，通过添加惩罚项来防止模型过拟合。

在使用 Python 进行机器学习时，整个过程一般遵循以下步骤：

1. **导入必要的库** - 例如，NumPy、Pandas 和 Scikit-learn。
 2. **加载和准备数据** - 数据是机器学习的核心。你需要加载数据并进行必要的预处理（例如数据清洗、缺失值填补等）。
 3. **选择模型和算法** - 根据任务选择适合的机器学习算法（如线性回归、决策树等）。
 4. **训练模型** - 使用训练集数据来训练模型。
 5. **评估模型** - 使用测试集评估模型的准确性，并根据评估结果优化模型。
 6. **调整模型和超参数** - 根据评估结果调整模型的超参数，进一步优化模型性能。
-

神经网络

神经网络（Neural Network，NN）是一种模拟人脑神经网络的机器学习模型，由多个神经元（Neuron）组成，每个神经元接受多个输入，并输出一个值。

- 神经元：输入、权重、偏置、激活函数、输出
$$y = f(x_1w_1 + x_2w_2 + \dots + x_nw_n + b)$$
- 激活函数：Sigmoid、ReLU、Tanh、Softmax、...
- 网络结构（前馈网络）：全连接层、卷积层、池化层、...

PyTorch

PyTorch 是一个开源的深度学习框架，以其灵活性和动态计算图而广受欢迎。

安装

- Python 3.10+
- 包管理器 pip/conda
- 虚拟环境 venv/conda env
- [PyTorch 官方网站](#)

NOTE: Latest PyTorch requires Python 3.9 or later.

PyTorch Build	Stable (2.6.0)			Preview (Nightly)	
Your OS	Linux		Mac		Windows
Package	Conda	Pip		LibTorch	Source
Language	Python			C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.4	CUDA 12.6	ROCm 6.2.4	CPU
Run this Command:	pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu124				

PyTorch 主要有以下几个基础概念：张量（Tensor）、自动求导（Autograd）、神经网络模块（nn.Module）、优化器（optim）等。

- **张量 (Tensor)** : PyTorch 的核心数据结构, 支持多维数组, 并可以在 CPU 或 GPU 上进行加速计算。
- **自动求导 (Autograd)** : PyTorch 提供了自动求导功能, 可以轻松计算模型的梯度, 便于进行反向传播和优化。
- **神经网络 (nn.Module)** : PyTorch 提供了简单且强大的 API 来构建神经网络模型, 可以方便地进行前向传播和模型定义。
- **优化器 (Optimizers)** : 使用优化器 (如 Adam、SGD 等) 来更新模型的参数, 使得损失最小化。
- **设备 (Device)** : 可以将模型和张量移动到 GPU 上以加速计算。

张量

```
import torch

# 创建一个 2x3 的全 0 张量
a = torch.zeros(2, 3)
print(a)

# 创建一个 2x3 的全 1 张量
b = torch.ones(2, 3)
print(b)

# 创建一个 2x3 的随机数张量
c = torch.randn(2, 3)
print(c)

# 从 NumPy 数组创建张量
import numpy as np
numpy_array = np.array([[1, 2], [3, 4]])
tensor_from_numpy = torch.from_numpy(numpy_array)
print(tensor_from_numpy)

# 在指定设备（CPU/GPU）上创建张量
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
d = torch.randn(2, 3, device=device)
print(d)

# 张量相加
e = torch.randn(2, 3)
f = torch.randn(2, 3)
print(e + f)

# 逐元素乘法
print(e * f)

# 张量的转置
g = torch.randn(3, 2)
print(g.t()) # 或者 g.transpose(0, 1)

# 张量的形状
print(g.shape) # 返回形状
```

自动求导

- 反向传播

```
# 反向传播，计算梯度
out.backward()
```

```
# 查看 x 的梯度
print(x.grad)
```

- 停用梯度

```
# 使用 torch.no_grad() 禁用梯度计算
with torch.no_grad():
    y = x * 2
    print(y)
    print(y.requires_grad) # False
```

神经网络模块

PyTorch 提供了一个非常方便的接口来构建神经网络模型，即 `torch.nn.Module`。

我们可以继承 `nn.Module` 类并定义自己的网络层。

```
import torch.nn as nn
import torch.optim as optim

# 定义一个简单的全连接神经网络
class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(2, 2) # 输入层到隐藏层
        self.fc2 = nn.Linear(2, 1) # 隐藏层到输出层

    def forward(self, x):
        x = torch.relu(self.fc1(x)) # ReLU 激活函数
        x = self.fc2(x)
        return x

# 创建网络实例
model = SimpleNN()

# 打印模型结构
print(model)
```

训练过程：

1. **前向传播 (Forward Propagation)** : 在前向传播阶段, 输入数据通过网络层传递, 每层应用权重和激活函数, 直到产生输出。
2. **计算损失 (Calculate Loss)** : 根据网络的输出和真实标签, 计算损失函数的值。
3. **反向传播 (Backpropagation)** : 反向传播利用自动求导技术计算损失函数关于每个参数的梯度。
4. **参数更新 (Parameter Update)** : 使用优化器根据梯度更新网络的权重和偏置。
5. **迭代 (Iteration)** : 重复上述过程, 直到模型在训练数据上的性能达到满意的水平。

优化器

```
# 定义优化器 (使用 Adam 优化器)
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 训练步骤
optimizer.zero_grad() # 清空梯度
loss.backward() # 反向传播
optimizer.step() # 更新参数
```

训练

出于设备条件、训练成本、模型性能和实时推理效率的综合考虑, 选择使用轻量级模型进行训练。考虑使用的模型:

- Mobilenetv4(large_100)
- ViT(base-patch16-224-in21k)
- BeiT(large-patch16-384)
- YOLOv11-m
- ...

MobileNetV4

<https://blog.csdn.net/hhhhhhhhhwww/article/details/139452661>

<https://blog.csdn.net/hhhhhhhhhwww/article/details/139554114>

Layer (type:depth-idx)	Output Shape	Param #
└─Conv2d: 1-1	[-1, 32, 112, 112]	864
└─BatchNormAct2d: 1-2	[-1, 32, 112, 112]	--
└─Identity: 2-1	[-1, 32, 112, 112]	--
└─ReLU: 2-2	[-1, 32, 112, 112]	--
└─Sequential: 1-3	[-1, 960, 7, 7]	--
└─Sequential: 2-3	[-1, 48, 56, 56]	--
└─EdgeResidual: 3-1	[-1, 48, 56, 56]	43,360
└─Sequential: 2-4	[-1, 80, 28, 28]	--
└─UniversalInvertedResidual: 3-2	[-1, 80, 28, 28]	30,832
└─UniversalInvertedResidual: 3-3	[-1, 80, 28, 28]	28,720
└─Sequential: 2-5	[-1, 160, 14, 14]	--
└─UniversalInvertedResidual: 3-4	[-1, 160, 14, 14]	130,320
└─UniversalInvertedResidual: 3-5	[-1, 160, 14, 14]	215,200
└─UniversalInvertedResidual: 3-6	[-1, 160, 14, 14]	215,200
└─UniversalInvertedResidual: 3-7	[-1, 160, 14, 14]	225,440
└─UniversalInvertedResidual: 3-8	[-1, 160, 14, 14]	215,200
└─UniversalInvertedResidual: 3-9	[-1, 160, 14, 14]	208,160
└─UniversalInvertedResidual: 3-10	[-1, 160, 14, 14]	103,360
└─UniversalInvertedResidual: 3-11	[-1, 160, 14, 14]	208,160
└─Sequential: 2-6	[-1, 256, 7, 7]	--
└─UniversalInvertedResidual: 3-12	[-1, 256, 7, 7]	432,032
└─UniversalInvertedResidual: 3-13	[-1, 256, 7, 7]	561,408
└─UniversalInvertedResidual: 3-14	[-1, 256, 7, 7]	557,312
└─UniversalInvertedResidual: 3-15	[-1, 256, 7, 7]	557,312
└─UniversalInvertedResidual: 3-16	[-1, 256, 7, 7]	526,848
└─UniversalInvertedResidual: 3-17	[-1, 256, 7, 7]	529,664
└─UniversalInvertedResidual: 3-18	[-1, 256, 7, 7]	280,320
└─UniversalInvertedResidual: 3-19	[-1, 256, 7, 7]	561,408
└─UniversalInvertedResidual: 3-20	[-1, 256, 7, 7]	526,848
└─UniversalInvertedResidual: 3-21	[-1, 256, 7, 7]	526,848
└─UniversalInvertedResidual: 3-22	[-1, 256, 7, 7]	270,592
└─Sequential: 2-7	[-1, 960, 7, 7]	--
└─ConvBnAct: 3-23	[-1, 960, 7, 7]	247,680
└─SelectAdaptivePool2d: 1-4	[-1, 960, 1, 1]	--
└─AdaptiveAvgPool2d: 2-8	[-1, 960, 1, 1]	--
└─Identity: 2-9	[-1, 960, 1, 1]	--
└─Conv2d: 1-5	[-1, 1280, 1, 1]	1,228,800
└─BatchNormAct2d: 1-6	[-1, 1280, 1, 1]	--
└─Identity: 2-10	[-1, 1280, 1, 1]	--
└─ReLU: 2-11	[-1, 1280, 1, 1]	--
└─Identity: 1-7	[-1, 1280, 1, 1]	--
└─Flatten: 1-8	[-1, 1280]	--
└─Linear: 1-9	[-1, 154]	197,274

```
=====
Total params: 8,629,162
Trainable params: 8,629,162
Non-trainable params: 0
Total mult-adds (M): 187.57
=====
```

```
Input size (MB): 0.57
Forward/backward pass size (MB): 7.64
Params size (MB): 32.92
Estimated Total Size (MB): 41.14
```

YOLOv11

环境

```
# Install the ultralytics package from PyPI
pip install ultralytics
```

数据集

对于 Ultralytics YOLO 分类任务，数据集必须在 root 目录下的特定分割目录结构中组织，以方便适当的训练、测试和可选验证过程。此结构包括用于训练（train）和测试（test）阶段的单独目录，以及可选的验证目录（val）。

每个这些目录应包含数据集中每个类的一个子目录。子目录以相应的类名命名，并包含该类的所有图像。确保每个图像文件具有唯一的名称，并以 JPEG 或 PNG 等通用格式存储。

```
cifar-10-/  
|-- train/  
|   |-- airplane/  
|   |-- automobile/  
|   |-- bird/  
|   ...  
|-- test/  
|   |-- airplane/  
|   |-- automobile/  
|   |-- bird/  
|   ...  
|-- val/ (optional)  
|   |-- airplane/  
|   |-- automobile/  
|   |-- bird/  
|   ...
```

Directory Structure: Organize your images into folders, with each folder name representing a class label 1

Image Processing: YOLO11 will automatically handle image resizing during training based on the `imgsz` parameter you set. Your original images can be kept at their native resolution

训练

变量	默认值	描述
model	None	Path to the model file for training.
data	None	Path to the dataset configuration file (e.g., coco8.yaml).
epochs	100	Total number of training epochs.
batch	16	Batch size, adjustable as integer or auto mode.
imgsz	640	Target image size for training.
device	None	Computational device(s) for training like cpu, 0, 0,1, or mps.
save	True	Enables saving of training checkpoints and final model weights.

- 超参数
- log (tensorboard)
- 模型保存

```
yolo classify resume train data=path/to/data model=yolo11n-cls.pt epochs=150 imgsz=640 patience
```

可视化

- tensorboard

```
tensorboard --logdir ultralytics/runs # replace with 'runs' directory
```

ViT

加载模型

构建数据集

https://huggingface.co/docs/datasets/create_dataset

```
DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 32227
  })
  validation: Dataset({
    features: ['image', 'label'],
    num_rows: 6909
  })
  test: Dataset({
    features: ['image', 'label'],
    num_rows: 6988
  })
})
dict_keys(['image', 'label'])
```

数据预处理

训练

```
pip install git+https://github.com/huggingface/accelerate
```